

CO2 - Assessment Tool-3

Compiler Design Scenario Based Task

J. Deepthi
19as25343
CSA1402

1) Scenario: Parsing Table Conflict in LL(1)

In an LL(1) Parser, the Parsing table is constructed using the First and Follow set of the grammar. The first set terminals determines that can appear at the beginning of strings derived from a production, while the follow set contains terminals that can appear immediately after a non-terminal. During parsing table construction, each production is placed in the table based on the sets.

A conflict occurs when more than one production is entered into the same table cell for a non-terminal and an input symbol. This usually happens because the grammar contains left recursion, common prefixes (left factoring problem), or is ambiguous. As a result, the parser cannot uniquely determine which production should be applied.

Since LL(1) Parsing is deterministic, every table entry must contain only one production. Multiple entries make the parser non-deterministic and may lead to incorrect parsing or parser failure.

Decision:- The grammar should be modified by eliminating left recursion, performing left factoring, and ensuring that the first & follow set do not overlap. This converts the grammar into a valid LL(1) grammar with a conflict-free parsing table.

2) Scenario: Incorrect Parse Tree Generation.
The syntax analyzer constructs a parse tree by applying grammar productions according to the input tokens. For the expression $id + id - id$, the parse tree determines the order in which operations are evaluated. If the grammar does not specify operator associativity, the expression may be grouped incorrectly. For example, it may be interpreted as $id + (id - id)$ instead of the correct $(id + id) - id$. This results in incorrect evaluation because the operators are not associated properly.

Grammar rules directly influence the structure of the parse tree. A grammar that does not enforce associativity allows multiple valid parse trees for the same expression.

Decision: The grammar should enforce left associativity for addition and subtraction.

For example:-

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow id$$

This ensures expressions are evaluated from left to right, producing the correct parse tree and accurate results.

3) Scenario: Invalid Expression Handling

The lexical analyzer correctly identifies the tokens in the expression $a * b$ as identifiers and operators. However, the syntax analyzer checks whether these tokens follow the grammar rules.

The error occurs because the grammar expects an operand after the $+$ operator, but another operator ($*$) appears instead. Therefore, the token sequence violates the grammar, making it impossible to construct a valid tree. In predictive parsing (LL), the parser detects error when no valid production exists in the

passing table for the current input symbol. In shift-reduce parsing (LR), the Parser detects the error when no valid shift or reduce action is available. Without proper recovery, the Parser stops after reporting the error.

Decision:-

The Parser should implement error detection and recovery techniques, such as Panic-mode recovery, Phrase-level recovery, or Synchronization using follow sets. These methods allow the Parser to continue analyzing the remaining input while providing meaningful error messages.

4) Scenario: Ambiguity in Conditional Statements.

The grammar

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow a$

is ambiguous because the Parser cannot determine which is statement or else belongs to. This is known as the dangling else problem.

When constructing the Parsing table, conflicts arise because both productions begin with the same prefix:

If E then S
The parser cannot decide whether to reduce using the first production (or) continue parsing the second production containing else.

This ambiguity affects deterministic parsing by introducing parsing conflicts and producing multiple possible parse trees.

Decision: The grammar should be rewritten to distinguish matched and unmatched statements.

for example:

- $S \rightarrow M \mid U$
- $M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid a$
- $U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U$

Alternatively, parser generators often resolve the ambiguity by associating else with the nearest unmatched if, which is the standard rule used in most programming languages.

5) Scenario: operator precedence conflict.

The grammar

- $E \rightarrow E + E$, • $E \rightarrow E * E$, • $E \rightarrow id$

is ambiguous because it does not define operator precedence (or) associativity.

For the input $id + id * id$, two different parse trees are possible:

- $(id + id) * id$
- $id + (id * id)$

The correct evaluation requires multiplication to have higher precedence than addition. Since the grammar does not specify this rule, the parser cannot determine the correct parse tree. Operator precedence and associativity are essential to ensure expressions are evaluated correctly.

Decision:- The grammar should be rewritten to enforce precedence, for example:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow id$

Here:

- Multiplication has higher precedence than addition.
- Addition and Multiplication are left associative.

→ Alternatively, Parser generators can use precedence and associativity declarations to resolve such conflicts automatically.